CRAFTIFAI PHYSICAL AI HACKATHON | TRACK A: PERCEPTION

SentinelEdge

Smart Traffic Analytics

Physical AI that sees traffic, decides counts, and acts through data.

SEE > DECIDE > ACT

PROJECT REPORT **Problem 4: Traffic and Crosswalk Analytics (PipeGen)**

TEAM **ZexTech team - 6e656f84**

AUTHOR **Zaheer Rachakula**

INSTITUTION **Vellore Institute of Technology (VIT), Vellore**

DATE **18 October 2026**

GITHUB https://github.com/ZAHEER-R/SentinelEdge

LIVE DEMO https://zaheer-traffic-analyzer.streamlit.app/

FILES https://drive.google.com/drive/folders/1j3363q5c5upAvIQRAR2wT30OcZC...

1 Project Overview

SentinelEdge turns a **video file, webcam or CCTV/RTSP stream** into live, per-class, directional vehicle counts. It follows the Physical AI cycle: **SEE** (YOLOv8 detection and ByteTrack tracking), **DECIDE** (line-crossing UP/DOWN counter) and **ACT** (JSON and MQTT outputs that a FirmGen board or other system can react to).

Item	Detail
Team	ZexTech team - 6e656f84
Author	Zaheer Rachakula, Integrated M.Tech (CSE), VIT Vellore
Event / Track	CraftifAI Physical AI Hackathon, Track A (Perception)
Problem	4. Traffic and Crosswalk Analytics (PipeGen): "Cities lack live counts of pedestrians, cyclists and cars at crossings."
GitHub	https://github.com/ZAHEER-R/SentinelEdge
Live demo	https://zaheer-traffic-analyzer.streamlit.app/
Project files (Drive)	Open Google Drive folder
Project form	https://forms.gle/eJyeCKCG2UH757QZ8
PipeGen downloads	https://developer.craftifai.com/dashboard?tab=downloads

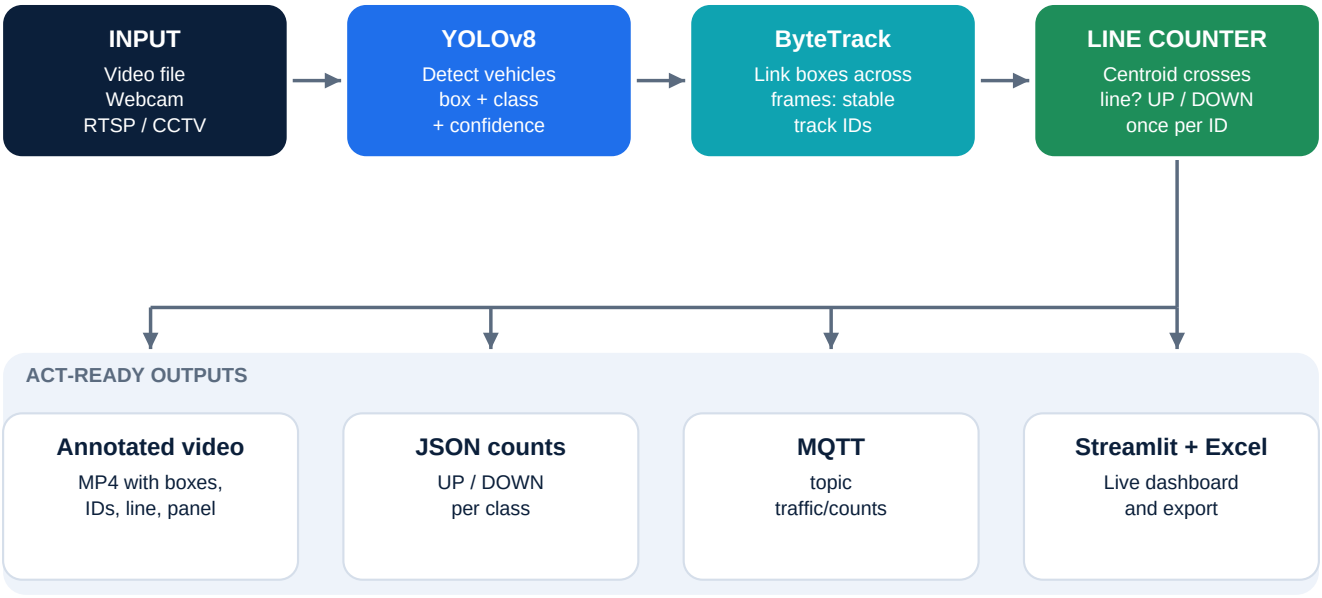
2 Problem Statement

Cities and roads often have cameras, but they usually only show video. They do not automatically count vehicles by type and direction in real time. Manual counting is slow and error-prone. Without structured counts it is hard to:

- Measure congestion by direction
- Plan signals or lanes
- Trigger alerts or physical actions (lights, barriers, logs)

Current scope. Cars, trucks, buses, motorcycles and bicycles, counted by direction (UP / DOWN). **Pedestrian counting** is not implemented yet and is planned as future work to fully match the official problem.

3 System Architecture



Counting rule

- A blue horizontal line is drawn across the frame (default about 55% down from the top).
- Each tracked vehicle has a **centroid** (center of its bounding box).
- Centroid moves from above the line to below: counted **DOWN**. From below to above: counted **UP**.
- Each track ID is counted **only once**, so there is no double counting.

On-screen output

Green bounding boxes, class name with track ID (for example `car ID:12`), the blue counting line, and a top-left panel such as `UP Cars: 3 / DOWN Cars: 12 / UP Trucks: 0 / DOWN Trucks: 2`.

4 Technology Stack

Layer	Technology	Role
Detection	Ultralytics YOLOv8 (yolov8n.pt)	Detect vehicles in each frame
Tracking	ByteTrack (via model.track)	Stable IDs over time
Vision I/O	OpenCV	Read streams, draw overlays, write MP4
Counting	src/counter.py	Line-crossing UP/DOWN logic
Visualization	src/visualizer.py	Boxes, line, count panel
Messaging	paho-mqtt	Publish to traffic/counts
Dashboard	Streamlit (app.py)	Web UI and Excel export (openpyxl)
Config	PyYAML (config.yaml)	Thresholds, line position, classes
Edge target	CraftifAI PipeGen > TensorRT > DeepStream	GPU / Jetson production path

Python packages: ultralytics>=8.3.0, opencv-python-headless>=4.8.0, numpy>=1.24.0, torch>=2.0.0, torchvision>=0.15.0, PyYAML>=6.0, tqdm>=4.65.0, paho-mqtt>=2.0.0, streamlit>=1.40,<2, openpyxl>=3.1.0, lap>=0.5.12

5 Two Deployment Paths

Path	What it is	Status
CPU / Dashboard	Python + YOLOv8 + Streamlit. Works on any PC, used for demo and development, deployed publicly.	**Completed**
PipeGen / GPU	CraftifAI PipeGen > TensorRT > DeepStream. The official Track A edge route on NVIDIA GPU / Jetson.	**Pending** (hardware limit)

6 Step-by-Step Build Process

1. **Problem selection**
   Reviewed the hackathon problems and chose Traffic and Crosswalk Analytics under Track A (Perception). Goal: live per-class UP/DOWN vehicle counts.

2. **Environment setup**
   Created a virtual environment (python -m venv venv), activated it, then ran pip install -r requirements.txt. Python 3.10+ (3.11 recommended), about 2 GB for model download.

3. **Reference pipeline**
   YOLOv8 (yolov8n.pt) for detection, ByteTrack via model.track for IDs, custom line counter in src/counter.py, OpenCV overlays in src/visualizer.py.

4. **Input handling (main.py)**
   Three modes: video file, webcam, RTSP URL. Interactive menu plus flags such as --mode video --source traffic.mp4. Press q to stop.

5. **Actionable outputs**
   Annotated MP4 and JSON counts saved to output/, and an MQTT publisher (src/mqtt_publisher.py) sending to traffic/counts for closed-loop use with a FirmGen board.

6. **Configuration layer**
   Moved tunable values to config.yaml: model (name, confidence, device), classes.vehicle, counting.line_position (0-1), display colors and thickness, output options.

7. **Streamlit dashboard**
   Built app.py to upload and process traffic video, view counts and export to Excel. Tested locally with streamlit run app.py.

8. **Deployment**
   Pushed the project to GitHub and deployed the dashboard on Streamlit Community Cloud (live demo link above).

9. **PipeGen requirement**
   Wrote PIPEGEN_REQUIREMENT.md so the same pipeline can be compiled to TensorRT / DeepStream on a GPU machine.

10. **Documentation**
    Wrote README.md, PROJECT_DESCRIPTION.md, SUBMISSION_CHECKLIST.md and VERIFICATION_STATUS.md, with setup for Windows PowerShell, WSL and PipeGen.

11. **Version control**
    git init, git add ., git branch -M main, git commit, git remote add origin, git push -u origin main.

12. **Submission and feedback**
    Submitted the GitHub and live demo URLs and filled the FirmGen/PipeGen feedback forms with the compatibility screenshot as evidence.

7 Repository Layout

```
SentinelEdge/
  README.md
  PROJECT_DESCRIPTION.md
  PIPEGEN_REQUIREMENT.md      <- paste into PipeGen
  SUBMISSION_CHECKLIST.md
  VERIFICATION_STATUS.md
  requirements.txt
  config.yaml
  main.py                     <- CLI pipeline
  app.py                      <- Streamlit dashboard
  src/
    counter.py
    visualizer.py
    mqtt_publisher.py
  evidence/                   <- IR + TensorRT screenshots (to add)
  pipegen_export/             <- PipeGen export (to add)
  output/
  docs/
```

8 Project Status

Component	Status
Vehicle detection (YOLOv8)	**Completed**
Multi-object tracking (ByteTrack)	**Completed**
Directional UP / DOWN line counting	**Completed**
Annotated video + JSON counts	**Completed**
MQTT counts publisher	**Completed**
Streamlit intelligence dashboard	**Completed and deployed**
Excel export	**Completed**
Live public demo	**Completed**
PipeGen requirement text	**Ready**
PipeGen TensorRT + DeepStream export	**Pending (needs NVIDIA GPU)**
Pipeline IR + TensorRT evidence	**Pending (after GPU run)**
Pedestrian counting	**Future work**

9 PipeGen / GPU Path: Requirements and Limitation

Requirement	Notes
NVIDIA GPU	Driver 560.35 or higher
Docker	PipeGen runs in a DeepStream container
NVIDIA Container Toolkit	GPU access inside the container
socat	Required by the launcher
docker group	User added to avoid constant sudo
Graphical session	Live display
Disk	About 50 GB free on x86_64
OS	Linux x86_64, Jetson (JetPack 6+), or WSL2 with GPU

Limitation. The author's laptop did not meet these requirements (a compatibility screenshot has been submitted). The PipeGen / TensorRT / DeepStream step was therefore delayed rather than skipped, and a complete working CPU-path solution was built and deployed instead.

Planned steps on a compatible machine

- Verify the driver with nvidia-smi; install Docker, NVIDIA Container Toolkit and socat; add the user to the docker group.
- Install PipeGen v0.5.x and verify with pipegen --version.
- Create a PipeGen project and paste PIPEGEN_REQUIREMENT.md into the Requirement stage.
- Complete the five stages: Requirement, Pipeline IR, Model (TensorRT FP16/INT8), Compiler, Run.
- Export into pipegen_export/ and add evidence/pipeline_ir.png and evidence/tensorrt_speed_report.png.
- Commit and push to GitHub.

10 How to Run

```
# Windows PowerShell
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python main.py            # CLI
streamlit run app.py      # dashboard

# WSL / Linux
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python main.py
streamlit run app.py
```

No install needed: open the live demo at https://zaheer-traffic-analyzer.streamlit.app/.

11 Challenges Faced

- GPU prerequisites for PipeGen (driver 560.35+, Docker with NVIDIA Container Toolkit, socat, about 50 GB disk, graphical session) were hard to meet on a student laptop / WSL2 setup.
- Hardware compatibility only became clear after setup had begun.

- **Solution:** built and deployed a full CPU-path system so the perception and counting logic could still be demonstrated end to end.

12 Future Work

- Run PipeGen on a compatible NVIDIA GPU machine and produce the TensorRT/DeepStream export, Pipeline IR and TensorRT speed-report evidence.
- Connect MQTT output (traffic/counts) to a FirmGen board for closed-loop physical action when thresholds are reached.
- Add pedestrian detection and counting to fully match the official problem.
- Extend to crosswalk analytics and multiple counting lines.

13 Conclusion

SentinelEdge delivers the SEE and DECIDE stages of Physical AI (vehicle detection, tracking and directional per-class counting) and exposes ACT-ready outputs (JSON and MQTT). The CPU path is complete, deployed and publicly accessible. The GPU / PipeGen path is prepared and will be completed on compatible hardware.

Thanks to the Devnovate team, the Microsoft team and Rakesh sir for the opportunity.

ZexTech team - 6e656f84
Zaheer Rachakula, Integrated M.Tech (CSE), VIT Vellore
GitHub: https://github.com/ZAHEER-R/SentinelEdge
Demo: https://zaheer-traffic-analyzer.streamlit.app/
Files: Google Drive folder